

# Inżynieria danych

## Wykład 12: wykonywanie zapytań

Katarzyna Grygiel

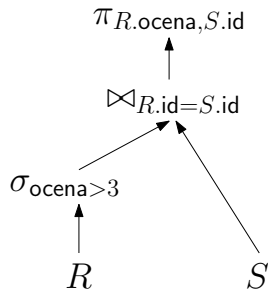
Semestr letni 2025/2026

## Zapytanie (na poziomie logicznym)

Zapytanie wyrażone w języku algebry relacyjnej można reprezentować za pomocą drzewa.

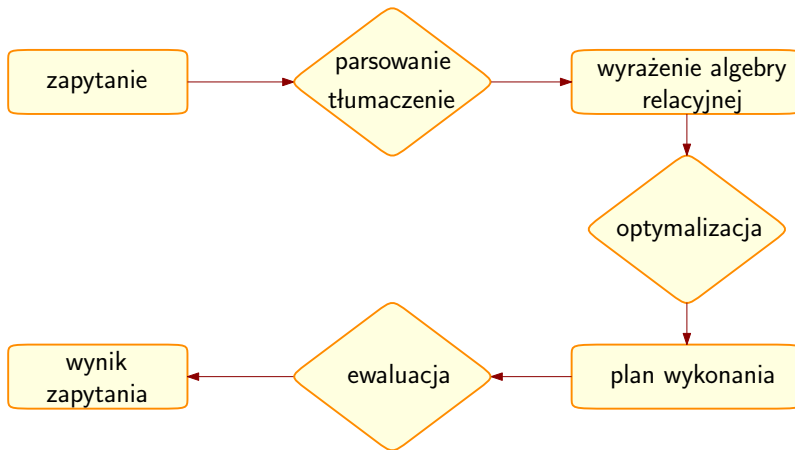
W liściach znajdują się relacje, natomiast w węzłach wewnętrznych operatory.

Dane przekazywane są w górę z liści do korzenia, a ostateczny wynik otrzymywany jest jako efekt działania operatora znajdującego się w korzeniu.



```
SELECT  R.ocena, S.id
FROM    R, S
WHERE   R.id = S.id
AND     R.ocena > 3;
```

## Zapytanie: co z nim zrobić?



# Parsowanie i tłumaczenie

Przed wykonaniem zapytania, system musi je przetłumaczyć do odpowiedniej postaci. W systemach relacyjnych jest ona oparta na algebrze relacyjnej.

Ten proces jest podobny do procesów przeprowadzanych przez parser lub kompilator. Sprawdzana jest składnia, obecność w bazie tabel i atrybutów, do których odwołuje się zapytanie, zgodność typów itd.

# Parsowanie i tłumaczenie

Przed wykonaniem zapytania, system musi je przetłumaczyć do odpowiedniej postaci. W systemach relacyjnych jest ona oparta na algebrze relacyjnej.

Ten proces jest podobny do procesów przeprowadzanych przez parser lub kompilator. Sprawdzana jest składnia, obecność w bazie tabel i atrybutów, do których odwołuje się zapytanie, zgodność typów itd.

System tworzy drzewo reprezentujące zapytanie, które następnie jest tłumaczone na wyrażenie algebry relacyjnej.

# Parsowanie i tłumaczenie

Przed wykonaniem zapytania, system musi je przetłumaczyć do odpowiedniej postaci. W systemach relacyjnych jest ona oparta na algebrze relacyjnej.

Ten proces jest podobny do procesów przeprowadzanych przez parser lub kompilator. Sprawdzana jest składnia, obecność w bazie tabel i atrybutów, do których odwołuje się zapytanie, zgodność typów itd.

System tworzy drzewo reprezentujące zapytanie, które następnie jest tłumaczone na wyrażenie algebry relacyjnej.

Jeśli w zapytaniu jest odwołanie do widoku, to podczas tej fazy wszystkie odniesienia do widoku również są zamieniane na wyrażenie algebry relacyjnej definiujące dany widok.

## Wiele wyrażen dla jednego zapytania

KaŹde zapytanie moŹe byc wyraŹone za pomocą wielu równowaŹnych wyraŹen algebry relacyjnej.

Reprezentacja za pomocą algebry relacyjnej tylko częściowo określa, jak powinno byc wykonane zapytanie, gdyŹ najczęściej istnieje wiele dróg ewaluacji.

Tym samym dla danego zapytania istnieje wiele różnych sposobów obliczenia wyniku.

## Wiele wyrażeń dla jednego zapytania

Każde zapytanie może być wyrażone za pomocą wielu równoważnych wyrażeń algebry relacyjnej.

Reprezentacja za pomocą algebry relacyjnej tylko częściowo określa, jak powinno być wykonane zapytanie, gdyż najczęściej istnieje wiele dróg ewaluacji.

Tym samym dla danego zapytania istnieje wiele różnych sposobów obliczenia wyniku.

**Przykład.** Zapytaniu

```
SELECT ocena FROM studenci WHERE ocena > 3;
```

mogą odpowiadać następujące wyrażenia:

$$\sigma_{\text{ocena} > 3}(\pi_{\text{ocena}}(\text{studenci})),$$
$$\pi_{\text{ocena}}(\sigma_{\text{ocena} > 3}(\text{studenci})).$$



## Jeden operator, wiele algorytmów

Aby wykonać jedno działanie pojawiające się w wyrażeniu algebry relacyjnej, można stosować wiele algorytmów.

Wykonanie selekcji  $\sigma_{ocena > 3}(studenci)$  może polegać na przeszukaniu wszystkich krotek lub wykorzystaniu odpowiedniego indeksu.

## Jeden operator, wiele algorytmów

Aby wykonać jedno działanie pojawiające się w wyrażeniu algebry relacyjnej, można stosować wiele algorytmów.

Wykonanie selekcji  $\sigma_{ocena > 3}(studenci)$  może polegać na przeszukaniu wszystkich krotek lub wykorzystaniu odpowiedniego indeksu.

Nie wystarczy zatem samo wyrażenie, potrzebne są również instrukcje, w jaki sposób każdy operator ma być ewaluowany. Takie instrukcje mogą zawierać informacje o zastosowaniu konkretnego algorytmu lub indeksu.

## Jeden operator, wiele algorytmów

Aby wykonać jedno działanie pojawiające się w wyrażeniu algebry relacyjnej, można stosować wiele algorytmów.

Wykonanie selekcji  $\sigma_{ocena > 3}(studenci)$  może polegać na przeszukaniu wszystkich krotek lub wykorzystaniu odpowiedniego indeksu.

Nie wystarczy zatem samo wyrażenie, potrzebne są również instrukcje, w jaki sposób każdy operator ma być ewaluowany. Takie instrukcje mogą zawierać informacje o zastosowaniu konkretnego algorytmu lub indeksu.

Plan wykonania zapytania będzie składał się z wyrażień algebry relacyjnej wraz z listą takich instrukcji.

Różne plany zapytań mają różne koszty. SZBD powinien zadbać o to, aby wybrać taki plan, który minimalizuje koszt wykonania zapytania.

# Koszt zapytania

Koszt planu wykonania zapytania może być mierzony według różnych kryteriów, m.in.

- liczby odczytów z dysku i zapisów na dysk,
- czasu pracy procesora,
- kosztu komunikacji między węzłami w przypadku rozproszonych systemów bazodanowych.

# Koszt zapytania

Koszt planu wykonania zapytania może być mierzony według różnych kryteriów, m.in.

- liczby odczytów z dysku i zapisów na dysk,
- czasu pracy procesora,
- kosztu komunikacji między węzłami w przypadku rozproszonych systemów bazodanowych.

Jeśli baza danych przechowywana jest na twardym dysku, to czas odczytu/zapisu zazwyczaj dominuje nad pozostałymi.

W związku z tym (na potrzeby tego wykładu) szacując koszt wykonania zapytania, bierzemy pod uwagę liczbę operacji I/O.

# Sortowanie

## Zapytania a sortowanie

Jeśli w zapytaniu pojawia się klauzula `ORDER BY`, to sortowanie pojawia się w sposób naturalny.

Czy w przypadku jej braku również opłaca się sortować krotki? Są one przecież elementami relacji, czyli ich kolejność teoretycznie nie ma znaczenia.

# Zapytania a sortowanie

Jeśli w zapytaniu pojawia się klauzula `ORDER BY`, to sortowanie pojawia się w sposób naturalny.

Czy w przypadku jej braku również opłaca się sortować krotki? Są one przecież elementami relacji, czyli ich kolejność teoretycznie nie ma znaczenia.

Okazuje się, że również w wielu innych sytuacjach sortowanie okazuje się pomocne, na przykład podczas:

- usuwania duplikatów (`DISTINCT`),
- ładowania zbiorczego przy tworzeniu B+-drzewa,
- wykonywania zapytań z funkcjami agregującymi (`GROUP BY`),
- wykonywania algorytmów realizujących złączenia (`JOIN`).



## Sortowanie: jaki algorytm wybrać?

Jeśli wszystkie dane, które chcemy posortować, mieszczą się w pamięci operacyjnej, to wystarczy skorzystać z jakiegokolwiek algorytmu sortującego (np. quicksort).

## Sortowanie: jaki algorytm wybrać?

Jeśli wszystkie dane, które chcemy posortować, mieszczą się w pamięci operacyjnej, to wystarczy skorzystać z jakiegokolwiek algorytmu sortującego (np. quicksort).

Problem pojawia się wtedy, gdy danych jest zbyt wiele i nie jesteśmy w stanie przechowywać ich wszystkich naraz w szybkiej pamięci.

## Sortowanie: jaki algorytm wybrać?

Jeśli wszystkie dane, które chcemy posortować, mieszczą się w pamięci operacyjnej, to wystarczy skorzystać z jakiegokolwiek algorytmu sortującego (np. quicksort).

Problem pojawia się wtedy, gdy danych jest zbyt wiele i nie jesteśmy w stanie przechowywać ich wszystkich naraz w szybkiej pamięci.

W takiej sytuacji należy zastosować taką metodę sortowania, który pozwoli zminimalizować liczbę operacji I/O (zmaksymalizuje liczbę sekwencyjnych dostępów).

Takim algorytmem, wykorzystywanym przez większość systemów bazodanowych, jest algorytm **zewnętrznego sortowania przez scalanie** (*external merge sort*).

# Zewnętrzne sortowanie przez scalanie

Algorytm działa jak standardowe sortowanie przez scalanie, ale ważne jest, co i kiedy przechowywane jest na dysku, a co w buforach pamięci operacyjnej.

# Zewnętrzne sortowanie przez scalanie

Algorytm działa jak standardowe sortowanie przez scalanie, ale ważne jest, co i kiedy przechowywane jest na dysku, a co w buforach pamięci operacyjnej.

Wyróżniamy dwie fazy:

- fazę **sortowania**, podczas której małe zbiory danych (mieszczące się w pamięci operacyjnej) są sortowane, a następnie zapisywane do pliku na dysku,
- fazę **scalania**, podczas której posortowane fragmenty pliku są scalane w jeden większy fragment.

# Zewnętrzne sortowanie przez scalanie

Algorytm działa jak standardowe sortowanie przez scalanie, ale ważne jest, co i kiedy przechowywane jest na dysku, a co w buforach pamięci operacyjnej.

Wyróżniamy dwie fazy:

- fazę **sortowania**, podczas której małe zbiory danych (mieszczące się w pamięci operacyjnej) są sortowane, a następnie zapisywane do pliku na dysku,
- fazę **scalania**, podczas której posortowane fragmenty pliku są scalane w jeden większy fragment.

Zakładamy, że plik z danymi do posortowania składa się z  $N$  stron, natomiast SZBD ma do dyspozycji  $B$  stron w pamięci operacyjnej.

# Zewnętrzne sortowanie przez scalanie: opis

Najprostsza wersja dotyczy łączenia dwóch posortowanych fragmentów – w tej sytuacji wystarczy  $B = 3$ .

przebieg nr 0:

- każda ze stron pliku jest wczytywana do pamięci i sortowana,
- posortowany fragment, zwany **serią** (*run*), przesyłany jest z powrotem na dysk.

# Zewnętrzne sortowanie przez scalanie: opis

Najprostsza wersja dotyczy łączenia dwóch posortowanych fragmentów – w tej sytuacji wystarczy  $B = 3$ .

przebieg nr 0:

- każda ze stron pliku jest wczytywana do pamięci i sortowana,
- posortowany fragment, zwany **serią** (*run*), przesyłany jest z powrotem na dysk.

przebiegi nr 1, 2, ...:

- rekurencyjnie scalane są pary serii, tworząc wyjściowe serie o długości dwa razy większej niż serie wejściowe, a następnie przesyłane są z powrotem na dysk,
- w buforze wykorzystuje się trzy strony: dwie dla danych wejściowych, jedną dla wyjściowych.



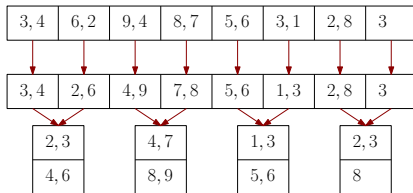
## Przykład

|      |      |      |      |      |      |      |   |
|------|------|------|------|------|------|------|---|
| 3, 4 | 6, 2 | 9, 4 | 8, 7 | 5, 6 | 3, 1 | 2, 8 | 3 |
|------|------|------|------|------|------|------|---|

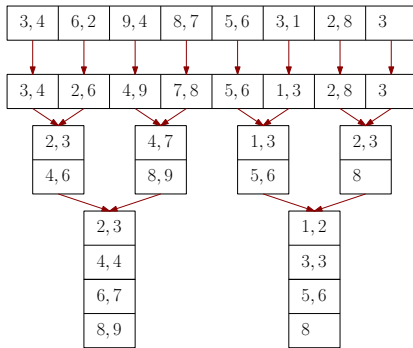
## Przykład

|      |      |      |      |      |      |      |   |
|------|------|------|------|------|------|------|---|
| 3, 4 | 6, 2 | 9, 4 | 8, 7 | 5, 6 | 3, 1 | 2, 8 | 3 |
| ↓    | ↓    | ↓    | ↓    | ↓    | ↓    | ↓    | ↓ |
| 3, 4 | 2, 6 | 4, 9 | 7, 8 | 5, 6 | 1, 3 | 2, 8 | 3 |

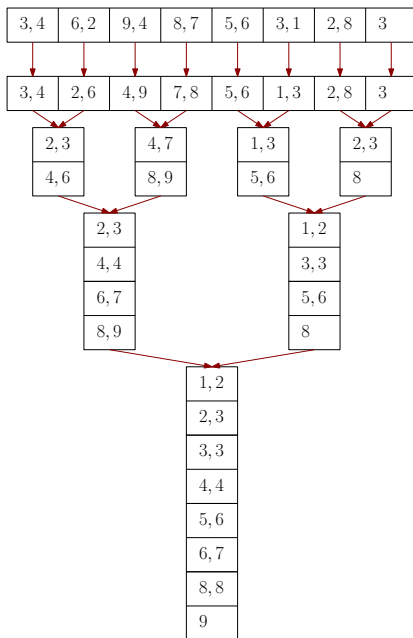
## Przykład



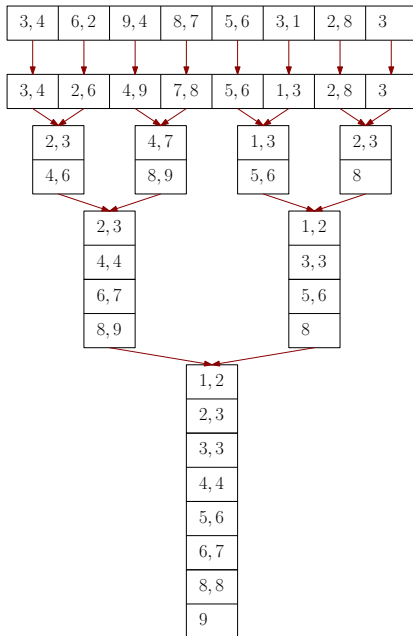
# Przykład



# Przykład



## Przykład



### Analiza kosztu

W każdym przebiegu czytamy i zapisujemy każdą ze stron do pliku.

Liczba przebiegów:

$$1 + \lceil \log_2 N \rceil.$$

Liczba operacji I/O:

$$2N \cdot (1 + \lceil \log_2 N \rceil).$$

## Więcej stron scalanych równocześnie

Nawet jeśli mamy do dyspozycji  $B > 3$  stron w buforze, to przy scalaniu tylko dwóch serii liczba odczytów się nie zmienia.

## Więcej stron scalanych równocześnie

Nawet jeśli mamy do dyspozycji  $B > 3$  stron w buforze, to przy scalaniu tylko dwóch serii liczba odczytów się nie zmienia.

W uogólnionej wersji zakładamy, że możemy scalać  $B - 1$  serii.

W zerowym przebiegu wykorzystujemy  $B$  stron bufora i zwracamy  $\lceil N/B \rceil$  uporządkowanych bloków.

W kolejnych przebiegach scalamy  $B - 1$  serii, a jedną stronę wykorzystujemy na przesyłanie posortowanych danych.



## Więcej stron scalanych równocześnie

Nawet jeśli mamy do dyspozycji  $B > 3$  stron w buforze, to przy scalaniu tylko dwóch serii liczba odczytów się nie zmienia.

W uogólnionej wersji zakładamy, że możemy scalać  $B - 1$  serii.

W zerowym przebiegu wykorzystujemy  $B$  stron bufora i zwracamy  $\lceil N/B \rceil$  uporządkowanych bloków.

W kolejnych przebiegach scalamy  $B - 1$  serii, a jedną stronę wykorzystujemy na przesyłanie posortowanych danych.

### Analiza kosztu

liczba przebiegów:

$$1 + \lceil \log_{B-1} \lceil N/B \rceil \rceil$$

liczba dostępów do dysku:

$$2N \cdot (1 + \lceil \log_{B-1} \lceil N/B \rceil \rceil).$$

## Przykład

Sortujemy 108 stron mając 5 stron w buforze, czyli  $N = 108$ ,  $B = 5$ .

Liczba przebiegów:  $1 + \lceil \log_{B-1} \lceil N/B \rceil \rceil = 1 + \lceil \log_4 22 \rceil = 4$ .

## Przykład

Sortujemy 108 stron mając 5 stron w buforze, czyli  $N = 108$ ,  $B = 5$ .

Liczba przebiegów:  $1 + \lceil \log_{B-1} \lceil N/B \rceil \rceil = 1 + \lceil \log_4 22 \rceil = 4$ .

przebieg nr 0:

Otrzymujemy  $\lceil 108/5 \rceil = 22$  posortowane serie, każda złożona z 5 stron, poza ostatnią, która ma 3 strony.

## Przykład

Sortujemy 108 stron mając 5 stron w buforze, czyli  $N = 108$ ,  $B = 5$ .

Liczba przebiegów:  $1 + \lceil \log_{B-1} \lceil N/B \rceil \rceil = 1 + \lceil \log_4 22 \rceil = 4$ .

przebieg nr 0:

Otrzymujemy  $\lceil 108/5 \rceil = 22$  posortowane serie, każda złożona z 5 stron, poza ostatnią, która ma 3 strony.

przebieg nr 1:

Otrzymujemy  $\lceil 22/4 \rceil = \lceil 22/4 \rceil = 6$  posortowanych serii, każda złożona z 20 stron, poza ostatnią, która ma 8 stron.

## Przykład

Sortujemy 108 stron mając 5 stron w buforze, czyli  $N = 108$ ,  $B = 5$ .

Liczba przebiegów:  $1 + \lceil \log_{B-1} \lceil N/B \rceil \rceil = 1 + \lceil \log_4 22 \rceil = 4$ .

przebieg nr 0:

Otrzymujemy  $\lceil 108/5 \rceil = 22$  posortowane serie, każda złożona z 5 stron, poza ostatnią, która ma 3 strony.

przebieg nr 1:

Otrzymujemy  $\lceil 22/4 \rceil = \lceil 22/4 \rceil = 6$  posortowanych serii, każda złożona z 20 stron, poza ostatnią, która ma 8 stron.

przebieg nr 2:

Otrzymujemy  $\lceil 6/4 \rceil = \lceil 6/4 \rceil = 2$  posortowane serie, pierwsza ma 80 stron, druga 28.

## Przykład

Sortujemy 108 stron mając 5 stron w buforze, czyli  $N = 108$ ,  $B = 5$ .

Liczba przebiegów:  $1 + \lceil \log_{B-1} \lceil N/B \rceil \rceil = 1 + \lceil \log_4 22 \rceil = 4$ .

przebieg nr 0:

Otrzymujemy  $\lceil 108/5 \rceil = 22$  posortowane serie, każda złożona z 5 stron, poza ostatnią, która ma 3 strony.

przebieg nr 1:

Otrzymujemy  $\lceil 22/4 \rceil = \lceil 22/4 \rceil = 6$  posortowanych serii, każda złożona z 20 stron, poza ostatnią, która ma 8 stron.

przebieg nr 2:

Otrzymujemy  $\lceil 6/4 \rceil = \lceil 6/4 \rceil = 2$  posortowane serie, pierwsza ma 80 stron, druga 28.

przebieg nr 3:

Otrzymujemy posortowany plik mający 108 stron.

## Sortowanie z wykorzystaniem indeksów

Jeżeli dla tabeli, którą chcemy posortować, jest założony indeks dla odpowiednich atrybutów, to możemy go oczywiście użyć do sortowania.

# Sortowanie z wykorzystaniem indeksów

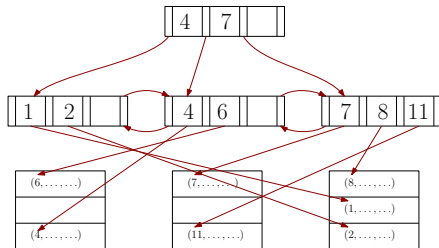
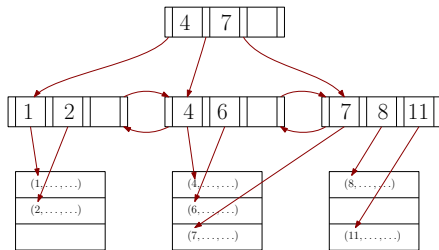
Jeżeli dla tabeli, którą chcemy posortować, jest założony indeks dla odpowiednich atrybutów, to możemy go oczywiście użyć do sortowania.

To, czy warto skorzystać z takiego rozwiązania, zależy od rodzaju indeksu.

- Jeśli indeks jest sklastrowany, czyli dane są przechowywane w odpowiedniej kolejności na dysku, to użycie go będzie zazwyczaj szybsze niż sortowanie zewnętrzne.
- Jeśli indeks nie jest sklastrowany, to skorzystanie z niego spowoduje, że liczba operacji I/O będzie porównywalna z liczbą I/O potrzebnych do odczytania każdej krotki z osobna.



## Sortowanie z wykorzystaniem indeksów



# Agregacja

## Sortowanie w agregacji

```
SELECT DISTINCT kurs  
FROM      studenci  
WHERE     ocena in (3, 4)  
ORDER BY  kurs;
```

| <i>studenci</i> |      |       |
|-----------------|------|-------|
| student         | kurs | ocena |
| 17              | 2    | 3     |
| 22              | 3    | 2     |
| 51              | 2    | 4     |
| 73              | 1    | 4     |
| 80              | 3    | 3     |
| 97              | 3    | 5     |

## Sortowanie w agregacji

```
SELECT DISTINCT kurs
FROM      studenci
WHERE     ocena in (3, 4)
ORDER BY  kurs;
```

| <i>studenci</i> |      |       |
|-----------------|------|-------|
| student         | kurs | ocena |
| 17              | 2    | 3     |
| 22              | 3    | 2     |
| 51              | 2    | 4     |
| 73              | 1    | 4     |
| 80              | 3    | 3     |
| 97              | 3    | 5     |

Filtrujemy, wybieramy kolumnę, sortujemy, usuwamy duplikaty:

| student | kurs | ocena |   | kurs |   | kurs |   | kurs |
|---------|------|-------|---|------|---|------|---|------|
| 17      | 2    | 3     |   | 2    |   | 1    |   | 1    |
| 51      | 2    | 4     | → | 2    | → | 2    | → | 2    |
| 73      | 1    | 4     |   | 1    |   | 2    |   | 3    |
| 80      | 3    | 3     |   | 3    |   | 3    |   |      |

## A jeśli nie ma ORDER BY?

W przypadku zapytania

```
SELECT DISTINCT kurs  
FROM   studenci  
WHERE  ocena in (3, 4);
```

sortowanie wprowadza dodatkowy koszt, a od ostatecznego wyniku nie oczekujemy uporządkowania.

## A jeśli nie ma ORDER BY?

W przypadku zapytania

```
SELECT DISTINCT kurs  
FROM   studenci  
WHERE  ocena in (3, 4);
```

sortowanie wprowadza dodatkowy koszt, a od ostatecznego wyniku nie oczekujemy uporządkowania.

Jeżeli nie zależy nam, aby wynik był posortowany, to zastosowanie funkcji haszujących często jest lepszym (szybszym) rozwiązaniem niż sortowanie.

Rozwiązania z funkcjami haszującymi wykorzystywane są przez większość systemów w zapytaniach z klauzulami DISTINCT oraz GROUP BY.

## Tablica z haszami

Podczas przeglądania danych z tabeli w pamięci operacyjnej tworzona jest tablica z haszami (istniejąca tylko w czasie wykonywania zapytania).

Dla każdej interesującej nas wartości nowej krotki sprawdzamy w tablicy, czy odpowiedni wpis już się tam znajduje.

## Tablica z haszami

Podczas przeglądania danych z tabeli w pamięci operacyjnej tworzona jest tablica z haszami (istniejąca tylko w czasie wykonywania zapytania).

Dla każdej interesującej nas wartości nowej krotki sprawdzamy w tablicy, czy odpowiedni wpis już się tam znajduje. Jeśli tak, to:

- przy wykonywaniu DISTINCT odrzucamy nową krotkę (mamy duplikat),
- przy wykonywaniu GROUP BY aktualizujemy wartość funkcji agregującej.



## Tablica z haszami

Podczas przeglądania danych z tabeli w pamięci operacyjnej tworzona jest tablica z haszami (istniejąca tylko w czasie wykonywania zapytania).

Dla każdej interesującej nas wartości nowej krotki sprawdzamy w tablicy, czy odpowiedni wpis już się tam znajduje. Jeśli tak, to:

- przy wykonywaniu DISTINCT odrzucamy nową krotkę (mamy duplikat),
- przy wykonywaniu GROUP BY aktualizujemy wartość funkcji agregującej.

Jeżeli tablica z haszami mieści się w pamięci, to powyższa procedura jest prosta.

Problem pojawia się w przypadku zbyt wielu danych, ponieważ musimy skorzystać wtedy z przestrzeni dyskowej.

# Haszowanie w agregacji: hash aggregate

Założmy, że mamy do dyspozycji  $B$  stron w buforze.

Algorytm **hash aggregate** składa się z dwóch faz:

- fazy **podziału**, podczas której w oparciu o funkcję haszującą  $h_1$  krotki są przydzielane do  $B - 1$  kubełków (jeden kubełek może zajmować więcej niż jedną stronę),
- fazy **powtórnego haszowania**, podczas której dla zawartości każdego kubełka wyznaczonego przez  $h_1$  tworzona jest tablica z haszami w oparciu o funkcję  $h_2$  i wyliczana jest wartość funkcji agregującej.

# Haszowanie w agregacji: hash aggregate

Założmy, że mamy do dyspozycji  $B$  stron w buforze.

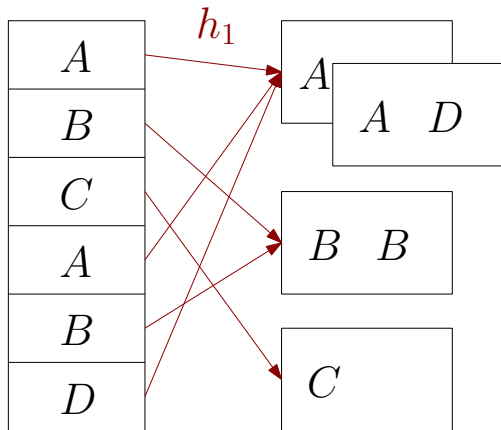
Algorytm **hash aggregate** składa się z dwóch faz:

- fazy **podziału**, podczas której w oparciu o funkcję haszującą  $h_1$  krotki są przydzielane do  $B - 1$  kubełków (jeden kubełek może zajmować więcej niż jedną stronę),
- fazy **powtórznego haszowania**, podczas której dla zawartości każdego kubełka wyznaczonego przez  $h_1$  tworzona jest tablica z haszami w oparciu o funkcję  $h_2$  i wyliczana jest wartość funkcji agregującej.

Powyższy algorytm działa przy założeniu, że zawartość każdego kubełka utworzonego funkcją  $h_1$  mieści się w pamięci operacyjnej.

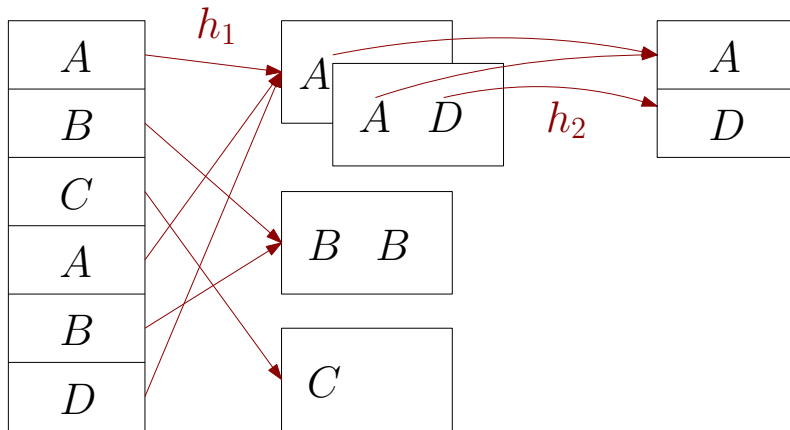
## Przykład

Mamy do dyspozycji 4 strony w buforze, czyli możemy utworzyć podział na 3 kubélki.



## Przykład

Mamy do dyspozycji 4 strony w buforze, czyli możemy utworzyć podział na 3 kubelki.



## Informacje w tablicy z haszami

Podczas fazy powtórnego haszowania w tablicy z haszami przechowujemy pary (klucz\_grupy, bieżąca\_wartość).

Kiedy odczytujemy zawartość nowej krotki, to:

- jeśli nie istnieje właściwy dla niej klucz\_grupy w tablicy z haszami, to dodajemy nową parę,
- jeśli istnieje odpowiedni klucz, to aktualizujemy odpowiadającą mu wartość funkcji agregującej.

## Informacje w tablicy z haszami

Podczas fazy powtórnego haszowania w tablicy z haszami przechowujemy pary (klucz\_grupy, bieżąca\_wartość).

Kiedy odczytujemy zawartość nowej krotki, to:

- jeśli nie istnieje właściwy dla niej klucz\_grupy w tablicy z haszami, to dodajemy nową parę,
- jeśli istnieje odpowiedni klucz, to aktualizujemy odpowiadającą mu wartość funkcji agregującej.

Aktualizacja w przypadku funkcji MAX, MIN, COUNT, SUM jest prosta, wystarczy pamiętać aktualne wartości.

Dla funkcji AVG przechowujemy parę wartości (COUNT, SUM).

Dla bardziej skomplikowanych funkcji konieczne może być przechowywanie dodatkowych składowych.

# Złączenia



# Algorytmy łączenia

Aby zapobiec redundancji i anomalom modyfikacji dokonujemy normalizacji tabel.

Kiedy jednak chcemy odczytać pełne dane, musimy z powrotem połączyć odpowiednie tabele. Wykorzystujemy do tego operator złączenia.

Ta operacja jest najczęściej najbardziej kosztowna spośród wszystkich pojawiających się w zapytaniu.

# Algorytmy łączenia

Aby zapobiec redundancji i anomalom modyfikacji dokonujemy normalizacji tabel.

Kiedy jednak chcemy odczytać pełne dane, musimy z powrotem połączyć odpowiednie tabele. Wykorzystujemy do tego operator złączenia.

Ta operacja jest najczęściej najbardziej kosztowna spośród wszystkich pojawiających się w zapytaniu.

Zakładamy do końca tych slajdów:

- łączymy tylko dwie tabele  $R$  i  $S$ ,
- złączenie jest równościowe,
- tabela  $R$  ma  $m$  krotek na  $M$  stronach,
- tabela  $S$  ma  $n$  krotek na  $N$  stronach.

## Wczesna i późna materializacja

```
SELECT R.ocena, S.id  
FROM   R, S  
WHERE  R.id = S.id  
AND    R.ocena > 3;
```

| <i>R</i> |      |       |
|----------|------|-------|
| id       | kurs | ocena |
| 13       | 321  | 2     |
| 13       | 123  | 4     |

| <i>S</i> |      |
|----------|------|
| id       | imię |
| 13       | Iwo  |
| 42       | Iga  |

## Wczesna i późna materializacja

```
SELECT R.ocena, S.id  
FROM   R, S  
WHERE  R.id = S.id  
AND    R.ocena > 3;
```

| <i>R</i> |      |       |
|----------|------|-------|
| id       | kurs | ocena |
| 13       | 321  | 2     |
| 13       | 123  | 4     |

| <i>S</i> |      |
|----------|------|
| id       | imię |
| 13       | Iwo  |
| 42       | Iga  |

- Zbieramy wszystkie informacje z tabel *R* i *S*. Kolejne operatory nie muszą już się odwoływać do ich zawartości.

| <i>R.id</i> | <i>R.kurs</i> | <i>R.ocena</i> | <i>S.id</i> | <i>S.imię</i> |
|-------------|---------------|----------------|-------------|---------------|
| 13          | 321           | 2              | 13          | Iwo           |
| 13          | 123           | 4              | 13          | Iwo           |

## Wczesna i późna materializacja

```
SELECT R.ocena, S.id
FROM   R, S
WHERE  R.id = S.id
AND    R.ocena > 3;
```

| <i>R</i> |      |       |
|----------|------|-------|
| id       | kurs | ocena |
| 13       | 321  | 2     |
| 13       | 123  | 4     |

| <i>S</i> |      |
|----------|------|
| id       | imię |
| 13       | lwo  |
| 42       | lga  |

- Zbieramy wszystkie informacje z tabel *R* i *S*. Kolejne operatory nie muszą już się odwoływać do ich zawartości.

| <i>R.id</i> | <i>R.kurs</i> | <i>R.ocena</i> | <i>S.id</i> | <i>S.imię</i> |
|-------------|---------------|----------------|-------------|---------------|
| 13          | 321           | 2              | 13          | lwo           |
| 13          | 123           | 4              | 13          | lwo           |

- Zbieramy tylko klucze i identyfikatory krotek.

| <i>R.id</i> | <i>R.ctid</i> | <i>S.id</i> | <i>S.ctid</i> |
|-------------|---------------|-------------|---------------|
| 13          | ...           | 13          | ...           |
| 13          | ...           | 13          | ...           |

## Zagnieżdżone pętle

Najprostszy algorytm łączenia to **nested-loop join**, czyli algorytm łączący dwie tabele z wykorzystaniem zagnieżdżonej pętli.

```
for each r in R
  for each s in S
    if match(r, s)
      return (r, s)
```

## Zagnieżdżone pętle

Najprostszy algorytm łączenia to **nested-loop join**, czyli algorytm łączący dwie tabele z wykorzystaniem zagnieżdżonej pętli.

```
for each r in R
  for each s in S
    if match(r, s)
      return (r, s)
```

Algorytm ten zwraca poprawny wynik, ale jest najwolniejszym z możliwych rozwiązań. Dla każdej krotki z  $R$  skanuje całą tabelę  $S$ .

## Zagnieżdżone pętle

Najprostszy algorytm łączenia to **nested-loop join**, czyli algorytm łączący dwie tabele z wykorzystaniem zagnieżdżonej pętli.

```
for each r in R
  for each s in S
    if match(r, s)
      return (r, s)
```

Algorytm ten zwraca poprawny wynik, ale jest najwolniejszym z możliwych rozwiązań. Dla każdej krotki z  $R$  skanuje całą tabelę  $S$ .

**Przykład.** Dla  $M = 1000$ ,  $m = 100\,000$ ,  $N = 500$ ,  $n = 40\,000$  koszt wynosi

$$M + (m \cdot N) = 50\,000\,100 \text{ I/O.}$$



## Zagnieżdżone pętle

Najprostszy algorytm łączenia to **nested-loop join**, czyli algorytm łączący dwie tabele z wykorzystaniem zagnieżdżonej pętli.

```
for each r in R
  for each s in S
    if match(r, s)
      return (r, s)
```

Algorytm ten zwraca poprawny wynik, ale jest najwolniejszym z możliwych rozwiązań. Dla każdej krotki z  $R$  skanuje całą tabelę  $S$ .

**Przykład.** Dla  $M = 1000$ ,  $m = 100\,000$ ,  $N = 500$ ,  $n = 40\,000$  koszt wynosi

$$M + (m \cdot N) = 50\,000\,100 \text{ I/O.}$$

Zakładając, że operacja I/O trwa 0,1 ms, otrzymujemy ok. 78 minut.

## Zagnieżdżone pętle

Najprostszy algorytm łączenia to **nested-loop join**, czyli algorytm łączący dwie tabele z wykorzystaniem zagnieżdżonej pętli.

```
for each r in R
  for each s in S
    if match(r, s)
      return (r, s)
```

Algorytm ten zwraca poprawny wynik, ale jest najwolniejszym z możliwych rozwiązań. Dla każdej krotki z  $R$  skanuje całą tabelę  $S$ .

**Przykład.** Dla  $M = 1000$ ,  $m = 100\,000$ ,  $N = 500$ ,  $n = 40\,000$  koszt wynosi

$$M + (m \cdot N) = 50\,000\,100 \text{ I/O.}$$

Zakładając, że operacja I/O trwa 0,1 ms, otrzymujemy ok. 78 minut.

Zamieniając  $R$  i  $S$ , otrzymujemy 40 000 500 I/O (ok. 66 minut).

## Trochę mniej zagnieżdżone pętle

Zamiast iterować się po każdej krotce z  $R$ , możemy wczytać całą stronę i dopiero wtedy iterować się po stronach z  $S$ . Jest to algorytm **block nested-loop join**.

## Trochę mniej zagnieżdżone pętle

Zamiast iterować się po każdej krotce z  $R$ , możemy wczytać całą stronę i dopiero wtedy iterować się po stronach z  $S$ . Jest to algorytm **block nested-loop join**.

```
for each block bR in R
  for each block bS in S
    for each r in bR
      for each s in bS
        if match(r, s)
          return (r, s)
```

Koszt maleje z  $M + (m \cdot N)$  do  $M + (M \cdot N)$ .

Dla tych samych danych co poprzednio otrzymujemy

$$1000 + (1000 \cdot 500) = 501\,000 \text{ I/O} \approx 50 \text{ sekund.}$$

## Mniej zagnieżdżonych pętli i bufor

Jeśli mamy do dyspozycji  $B$  buforów, to możemy  $B - 2$  spośród nich wykorzystać do przechowywania stron z relacji  $R$ , jedną stronę przeznaczyć dla relacji  $S$ , natomiast ostatnia strona posłuży nam do przechowywania danych wyjściowych.

## Mniej zagnieżdżonych pętli i bufor

Jeśli mamy do dyspozycji  $B$  buforów, to możemy  $B - 2$  spośród nich wykorzystać do przechowywania stron z relacji  $R$ , jedną stronę przeznaczyć dla relacji  $S$ , natomiast ostatnia strona posłuży nam do przechowywania danych wyjściowych.

```
for each B-2 blocks bR in R
  for each block bS in S
    for each r in bR
      for each s in bS
        if match(r, s)
          return (r, s)
```

W tej sytuacji koszt wynosi  $M + (\lceil M/(B - 2) \rceil \cdot N)$ .

Jeśli cała tabela  $R$  zmieści się w pamięci operacyjnej, to koszt wyniesie  $M + N$ .

Dla przykładowych danych przełoży się to na ok. 0,15 sekundy.

## Zagnieżdżone pętle i indeksy

Jeśli dla którejś z tabel zbudowany jest indeks (lub opłaca się nam go zbudować w celu utworzenia złączenia), to możemy wykorzystać algorytm **indexed nested-loop join**.

W tym przypadku dla każdej krotki relacji  $R$  sprawdzamy, czy odpowiadające jej wartości znajdują się w indeksie dla relacji  $S$ .

## Zagnieżdżone pętle i indeksy

Jeśli dla którejś z tabel zbudowany jest indeks (lub opłaca się nam go zbudować w celu utworzenia złączenia), to możemy wykorzystać algorytm **indexed nested-loop join**.

W tym przypadku dla każdej krotki relacji  $R$  sprawdzamy, czy odpowiadające jej wartości znajdują się w indeksie dla relacji  $S$ .

```
for each r in R
  for each s in Index(r[i], s[j])
    if match(r, s)
      return (r, s)
```



## Zagnieżdżone pętle i indeksy

Jeśli dla którejś z tabel zbudowany jest indeks (lub opłaca się nam go zbudować w celu utworzenia złączenia), to możemy wykorzystać algorytm **indexed nested-loop join**.

W tym przypadku dla każdej krotki relacji  $R$  sprawdzamy, czy odpowiadające jej wartości znajdują się w indeksie dla relacji  $S$ .

```
for each r in R
  for each s in Index(r[i], s[j])
    if match(r, s)
      return (r, s)
```

Koszt w tej sytuacji wynosi  $M + (m \cdot C)$ , gdzie  $C$  jest stałą zależną od rozmiaru indeksu i struktury danych, na której jest oparty.

## Zagnieżdżone pętle i indeksy

Jeśli dla którejś z tabel zbudowany jest indeks (lub opłaca się nam go zbudować w celu utworzenia złączenia), to możemy wykorzystać algorytm **indexed nested-loop join**.

W tym przypadku dla każdej krotki relacji  $R$  sprawdzamy, czy odpowiadające jej wartości znajdują się w indeksie dla relacji  $S$ .

```
for each r in R
  for each s in Index(r[i], s[j])
    if match(r, s)
      return (r, s)
```

Koszt w tej sytuacji wynosi  $M + (m \cdot C)$ , gdzie  $C$  jest stałą zależną od rozmiaru indeksu i struktury danych, na której jest oparty.

Na podstawie funkcji kosztu widać, że w przypadku istnienia dwóch indeksów (po jednym dla każdej z tabel), jako  $R$  opłaca się wziąć tabelę o mniejszej liczbie krotek.

## Sortowanie w służbie łączenia

Łącząc dwie tabele względem pewnego atrybutu, możemy posortować obie z nich według jego wartości. Sama procedura łączenia będzie wtedy szybka.

# Sortowanie w służbie łączenia

Łącząc dwie tabele względem pewnego atrybutu, możemy posortować obie z nich według jego wartości. Sama procedura łączenia będzie wtedy szybka.

Algorytm oparty na sortowaniu to **sort-merge join**.

Składa się z dwóch etapów:

- fazy **sortowania**, podczas której obie tabele sortujemy po atrybutach, po których łączymy (możliwe, że wykorzystamy do tego sortowanie zewnętrzne),
- fazy **scalania**, podczas której przechodzimy równolegle przez obie tabele i znajdujemy pasujące pary (jeśli wartości w obu tabelach nie są unikalne, być może nie unikniemy backtrackingu).

## Trochę dłuższy pseudokod

Poniższy pseudokod jest bardzo uproszczony, ma tylko na celu pokazanie idei działania algorytmu sort-merge join.

```
sort R, S on join keys
cursorR <- sortedR, cursorS <- sortedS
while cursorR and cursorS
  if cursorR > cursorS
    increment cursorS
  if cursorR < cursorS
    increment cursorR
  elif match(cursorR, cursorS)
    return (R(cursorR), S(cursorS))
    increment cursorS
```

## Przykład

```
SELECT S.ocena, R.id FROM R, S WHERE R.id = S.id AND S.ocena > 3;
```

| <i>R</i> |      |
|----------|------|
| id       | imię |
| 13       | Iwo  |
| 42       | Iga  |
| 73       | Ewa  |
| 87       | Jan  |
| 93       | Ola  |

| <i>S</i> |      |       |
|----------|------|-------|
| id       | kurs | ocena |
| 13       | 321  | 3     |
| 13       | 123  | 4     |
| 42       | 456  | 3     |
| 87       | 321  | 3     |

## Przykład

```
SELECT S.ocena, R.id FROM R, S WHERE R.id = S.id AND S.ocena > 3;
```

| <i>R</i> |    |      |
|----------|----|------|
|          | id | imię |
| →        | 13 | Iwo  |
|          | 42 | Iga  |
|          | 73 | Ewa  |
|          | 87 | Jan  |
|          | 93 | Ola  |

| <i>S</i> |    |      |       |
|----------|----|------|-------|
|          | id | kurs | ocena |
| →        | 13 | 321  | 3     |
|          | 13 | 123  | 4     |
|          | 42 | 456  | 3     |
|          | 87 | 321  | 3     |

## Przykład

```
SELECT S.ocena, R.id FROM R, S WHERE R.id = S.id AND S.ocena > 3;
```

| <i>R</i> |    |      |
|----------|----|------|
|          | id | imię |
| →        | 13 | Iwo  |
|          | 42 | Iga  |
|          | 73 | Ewa  |
|          | 87 | Jan  |
|          | 93 | Ola  |

| <i>S</i> |    |      |       |
|----------|----|------|-------|
|          | id | kurs | ocena |
| →        | 13 | 321  | 3     |
|          | 13 | 123  | 4     |
|          | 42 | 456  | 3     |
|          | 87 | 321  | 3     |

| <i>R.id</i> | <i>R.imię</i> | <i>S.id</i> | <i>S.kurs</i> | <i>S.ocena</i> |
|-------------|---------------|-------------|---------------|----------------|
| 13          | Iwo           | 13          | 321           | 3              |



## Przykład

```
SELECT S.ocena, R.id FROM R, S WHERE R.id = S.id AND S.ocena > 3;
```

| <i>R</i> |    |      |
|----------|----|------|
|          | id | imię |
| →        | 13 | Iwo  |
|          | 42 | Iga  |
|          | 73 | Ewa  |
|          | 87 | Jan  |
|          | 93 | Ola  |

| S |    |      |       |
|---|----|------|-------|
|   | id | kurs | ocena |
|   | 13 | 321  | 3     |
| → | 13 | 123  | 4     |
|   | 42 | 456  | 3     |
|   | 87 | 321  | 3     |

| <i>R.id</i> | <i>R.imię</i> | <i>S.id</i> | <i>S.kurs</i> | <i>S.ocena</i> |
|-------------|---------------|-------------|---------------|----------------|
| 13          | Iwo           | 13          | 321           | 3              |

## Przykład

```
SELECT S.ocena, R.id FROM R, S WHERE R.id = S.id AND S.ocena > 3;
```

| <i>R</i> |    |      |
|----------|----|------|
| <hr/>    |    |      |
|          | id | imię |
| <hr/>    |    |      |
| →        | 13 | Iwo  |
|          | 42 | Iga  |
|          | 73 | Ewa  |
|          | 87 | Jan  |
|          | 93 | Ola  |
| <hr/>    |    |      |

| <i>S</i> |    |      |       |
|----------|----|------|-------|
| <hr/>    |    |      |       |
|          | id | kurs | ocena |
| <hr/>    |    |      |       |
|          | 13 | 321  | 3     |
| →        | 13 | 123  | 4     |
|          | 42 | 456  | 3     |
|          | 87 | 321  | 3     |
| <hr/>    |    |      |       |

| <i>R.id</i> | <i>R.imię</i> | <i>S.id</i> | <i>S.kurs</i> | <i>S.ocena</i> |
|-------------|---------------|-------------|---------------|----------------|
| 13          | Iwo           | 13          | 321           | 3              |
| 13          | Iwo           | 13          | 123           | 4              |

## Przykład

```
SELECT S.ocena, R.id FROM R, S WHERE R.id = S.id AND S.ocena > 3;
```

| <i>R</i> |    |      |
|----------|----|------|
|          | id | imię |
| →        | 13 | Iwo  |
|          | 42 | Iga  |
|          | 73 | Ewa  |
|          | 87 | Jan  |
|          | 93 | Ola  |

| S |    |      |       |
|---|----|------|-------|
|   | id | kurs | ocena |
|   | 13 | 321  | 3     |
|   | 13 | 123  | 4     |
| → | 42 | 456  | 3     |
|   | 87 | 321  | 3     |

| <i>R.id</i> | <i>R.imię</i> | <i>S.id</i> | <i>S.kurs</i> | <i>S.ocena</i> |
|-------------|---------------|-------------|---------------|----------------|
| 13          | Iwo           | 13          | 321           | 3              |
| 13          | Iwo           | 13          | 123           | 4              |

## Przykład

```
SELECT S.ocena, R.id FROM R, S WHERE R.id = S.id AND S.ocena > 3;
```

|   | <i>R</i> |      |
|---|----------|------|
|   | id       | imię |
|   | 13       | Iwo  |
| → | 42       | Iga  |
|   | 73       | Ewa  |
|   | 87       | Jan  |
|   | 93       | Ola  |

|   | <i>S</i> |      |       |
|---|----------|------|-------|
|   | id       | kurs | ocena |
|   | 13       | 321  | 3     |
|   | 13       | 123  | 4     |
| → | 42       | 456  | 3     |
|   | 87       | 321  | 3     |

| <i>R.id</i> | <i>R.imię</i> | <i>S.id</i> | <i>S.kurs</i> | <i>S.ocena</i> |
|-------------|---------------|-------------|---------------|----------------|
| 13          | Iwo           | 13          | 321           | 3              |
| 13          | Iwo           | 13          | 123           | 4              |

## Przykład

```
SELECT S.ocena, R.id FROM R, S WHERE R.id = S.id AND S.ocena > 3;
```

| <i>R</i> |      |
|----------|------|
| id       | imię |
| 13       | Iwo  |
| → 42     | Iga  |
| 73       | Ewa  |
| 87       | Jan  |
| 93       | Ola  |

| <i>S</i> |      |       |
|----------|------|-------|
| id       | kurs | ocena |
| 13       | 321  | 3     |
| 13       | 123  | 4     |
| → 42     | 456  | 3     |
| 87       | 321  | 3     |

| <i>R.id</i> | <i>R.imię</i> | <i>S.id</i> | <i>S.kurs</i> | <i>S.ocena</i> |
|-------------|---------------|-------------|---------------|----------------|
| 13          | Iwo           | 13          | 321           | 3              |
| 13          | Iwo           | 13          | 123           | 4              |
| 42          | Iga           | 42          | 456           | 3              |

## Przykład

```
SELECT S.ocena, R.id FROM R, S WHERE R.id = S.id AND S.ocena > 3;
```

| <i>R</i> |      | <i>S</i> |      |       |
|----------|------|----------|------|-------|
| id       | imię | id       | kurs | ocena |
| 13       | Iwo  | 13       | 321  | 3     |
| → 42     | Iga  | 13       | 123  | 4     |
| 73       | Ewa  | 42       | 456  | 3     |
| 87       | Jan  | → 87     | 321  | 3     |
| 93       | Ola  |          |      |       |

| <i>R.id</i> | <i>R.imię</i> | <i>S.id</i> | <i>S.kurs</i> | <i>S.ocena</i> |
|-------------|---------------|-------------|---------------|----------------|
| 13          | Iwo           | 13          | 321           | 3              |
| 13          | Iwo           | 13          | 123           | 4              |
| 42          | Iga           | 42          | 456           | 3              |

## Przykład

```
SELECT S.ocena, R.id FROM R, S WHERE R.id = S.id AND S.ocena > 3;
```

| <i>R</i> |      | <i>S</i> |      |       |
|----------|------|----------|------|-------|
| id       | imię | id       | kurs | ocena |
| 13       | Iwo  | 13       | 321  | 3     |
| 42       | Iga  | 13       | 123  | 4     |
| → 73     | Ewa  | 42       | 456  | 3     |
| 87       | Jan  | → 87     | 321  | 3     |
| 93       | Ola  |          |      |       |

| <i>R.id</i> | <i>R.imię</i> | <i>S.id</i> | <i>S.kurs</i> | <i>S.ocena</i> |
|-------------|---------------|-------------|---------------|----------------|
| 13          | Iwo           | 13          | 321           | 3              |
| 13          | Iwo           | 13          | 123           | 4              |
| 42          | Iga           | 42          | 456           | 3              |

## Przykład

```
SELECT S.ocena, R.id FROM R, S WHERE R.id = S.id AND S.ocena > 3;
```

| <i>R</i> |      |
|----------|------|
| id       | imię |
| 13       | Iwo  |
| 42       | Iga  |
| 73       | Ewa  |
| → 87     | Jan  |
| 93       | Ola  |

| <i>S</i> |      |       |
|----------|------|-------|
| id       | kurs | ocena |
| 13       | 321  | 3     |
| 13       | 123  | 4     |
| 42       | 456  | 3     |
| → 87     | 321  | 3     |

| <i>R.id</i> | <i>R.imię</i> | <i>S.id</i> | <i>S.kurs</i> | <i>S.ocena</i> |
|-------------|---------------|-------------|---------------|----------------|
| 13          | Iwo           | 13          | 321           | 3              |
| 13          | Iwo           | 13          | 123           | 4              |
| 42          | Iga           | 42          | 456           | 3              |



## Przykład

```
SELECT S.ocena, R.id FROM R, S WHERE R.id = S.id AND S.ocena > 3;
```

| <i>R</i> |      |
|----------|------|
| id       | imię |
| 13       | Iwo  |
| 42       | Iga  |
| 73       | Ewa  |
| → 87     | Jan  |
| 93       | Ola  |

| <i>S</i> |      |       |
|----------|------|-------|
| id       | kurs | ocena |
| 13       | 321  | 3     |
| 13       | 123  | 4     |
| 42       | 456  | 3     |
| → 87     | 321  | 3     |

| <i>R.id</i> | <i>R.imię</i> | <i>S.id</i> | <i>S.kurs</i> | <i>S.ocena</i> |
|-------------|---------------|-------------|---------------|----------------|
| 13          | Iwo           | 13          | 321           | 3              |
| 13          | Iwo           | 13          | 123           | 4              |
| 42          | Iga           | 42          | 456           | 3              |
| 87          | Jan           | 87          | 321           | 3              |

## Przykład

```
SELECT S.ocena, R.id FROM R, S WHERE R.id = S.id AND S.ocena > 3;
```

| <i>R</i> |      |
|----------|------|
| id       | imię |
| 13       | Iwo  |
| 42       | Iga  |
| 73       | Ewa  |
| → 87     | Jan  |
| 93       | Ola  |

| <i>S</i> |      |       |
|----------|------|-------|
| id       | kurs | ocena |
| 13       | 321  | 3     |
| 13       | 123  | 4     |
| 42       | 456  | 3     |
| → 87     | 321  | 3     |

| <i>R.id</i> | <i>R.imię</i> | <i>S.id</i> | <i>S.kurs</i> | <i>S.ocena</i> |
|-------------|---------------|-------------|---------------|----------------|
| 13          | Iwo           | 13          | 321           | 3              |
| 13          | Iwo           | 13          | 123           | 4              |
| 42          | Iga           | 42          | 456           | 3              |
| 87          | Jan           | 87          | 321           | 3              |

## Przykład

```
SELECT S.ocena, R.id FROM R, S WHERE R.id = S.id AND S.ocena > 3;
```

| <i>R</i> |      |
|----------|------|
| id       | imię |
| 13       | Iwo  |
| 42       | Iga  |
| 73       | Ewa  |
| 87       | Jan  |
| 93       | Ola  |

| <i>S</i> |      |       |
|----------|------|-------|
| id       | kurs | ocena |
| 13       | 321  | 3     |
| 13       | 123  | 4     |
| 42       | 456  | 3     |
| 87       | 321  | 3     |

| <i>R.id</i> | <i>R.imię</i> | <i>S.id</i> | <i>S.kurs</i> | <i>S.ocena</i> |
|-------------|---------------|-------------|---------------|----------------|
| 13          | Iwo           | 13          | 321           | 3              |
| 13          | Iwo           | 13          | 123           | 4              |
| 42          | Iga           | 42          | 456           | 3              |
| 87          | Jan           | 87          | 321           | 3              |

## Koszt coraz mniejszy

Otrzymujemy następujące składowe koszty:

- sortowanie tabeli  $R$ :  $2M \cdot (\log M / \log B)$ ,
- sortowanie tabeli  $S$ :  $2N \cdot (\log N / \log B)$ ,
- scalanie:  $M + N$ .

Całkowity koszt algorytmu sort-merge join to suma wszystkich trzech powyższych kosztów.

## Koszt coraz mniejszy

Otrzymujemy następujące składowe koszty:

- sortowanie tabeli  $R$ :  $2M \cdot (\log M / \log B)$ ,
- sortowanie tabeli  $S$ :  $2N \cdot (\log N / \log B)$ ,
- scalanie:  $M + N$ .

Całkowity koszt algorytmu sort-merge join to suma wszystkich trzech powyższych kosztów.

Dla przykładowych danych otrzymujemy:

- sortowanie  $R$ : 3000 I/O,
- sortowanie  $S$ : 1350 I/O,
- scalanie: 1500 I/O,
- całkowity koszt: 5850 I/O.

Przekłada się to na ok. 0,59 sekundy.

## Podsumowanie łączenia przez sortowanie

W najgorszym przypadku może się tak zdarzyć, że w obu łączonych tabelach atrybuty, po których odbywa się łączenie, przyjmują tylko jedną wartość. Wtedy koszt łączenia algorytmem sort-merge join wynosi  $M \cdot N$  plus koszt sortowania obu tabel.

## Podsumowanie łączenia przez sortowanie

W najgorszym przypadku może się tak zdarzyć, że w obu łączonych tabelach atrybuty, po których odbywa się łączenie, przyjmują tylko jedną wartość. Wtedy koszt łączenia algorytmem sort-merge join wynosi  $M \cdot N$  plus koszt sortowania obu tabel.

Jest to jednak sytuacja nietypowa (nie powinna w ogóle mieć miejsca). Najczęściej łączymy tabelę po kluczach, z których jeden jest podstawowy, a drugi obcy i wskazuje na pierwszy. Wtedy mamy gwarancję, że wartości w pierwszej tabeli są unikalne.

## Podsumowanie łączenia przez sortowanie

W najgorszym przypadku może się tak zdarzyć, że w obu łączonych tabelach atrybuty, po których odbywa się łączenie, przyjmują tylko jedną wartość. Wtedy koszt łączenia algorytmem sort-merge join wynosi  $M \cdot N$  plus koszt sortowania obu tabel.

Jest to jednak sytuacja nietypowa (nie powinna w ogóle mieć miejsca). Najczęściej łączymy tabelę po kluczach, z których jeden jest podstawowy, a drugi obcy i wskazuje na pierwszy. Wtedy mamy gwarancję, że wartości w pierwszej tabeli są unikalne.

Wejściowe tabele w algorytmie sort-merge join można sortować zewnętrznie lub korzystając z indeksu.



## I na koniec: haszowanie

Jeśli krotki  $r \in R$  oraz  $s \in S$  spełniają warunek łączenia, to mają takie same wartości atrybutów, po których łączymy.

Jeśli zatem zastosujemy na nich funkcję haszującą, to trafimy do tych samych kubełków.

# I na koniec: haszowanie

Jeśli krotki  $r \in R$  oraz  $s \in S$  spełniają warunek łączenia, to mają takie same wartości atrybutów, po których łączymy.

Jeśli zatem zastosujemy na nich funkcję haszującą, to trafimy do tych samych kubełków.

Podstawowy algorytm **łączenia przez haszowanie** (*hash join*) składa się z dwóch etapów:

- fazy **tworzenia**, podczas której budujemy tablicę z haszami dla wartości atrybutów (wykorzystywanych przy złączeniu) relacji  $R$  w oparciu o pewną funkcję haszującą  $h_1$ ,
- fazy **próbkowania**, podczas której dla każdej krotki  $s \in S$  obliczamy wartości haszy  $h_1(s)$  i sprawdzamy, czy wartości odpowiednich atrybutów krotki  $s$  zgadzają się z wartościami krotek z  $R$ , które trafiły do tego samego kubełka co  $s$ .

## Co przechowujemy w tablicach z haszami?

W tablicach z haszami przechowujemy pary (klucz, wartość).

Kluczem są wartości atrybutów, po których łączymy.

Ale co przechowujemy jako wartość? To zależy przede wszystkim od tego, jakich informacji będą wymagać operatory leżące w drzewie wykonań powyżej operatora złączenia.

# Co przechowujemy w tablicach z haszami?

W tablicach z haszami przechowujemy pary (klucz, wartość).

Kluczem są wartości atrybutów, po których łączymy.

Ale co przechowujemy jako wartość? To zależy przede wszystkim od tego, jakich informacji będą wymagać operatory leżące w drzewie wykonania powyżej operatora złączenia.

Możemy przechowywać całą krotkę, aby nie odwoływać się już do samej tabeli. Wymaga to jednak odpowiednio więcej pamięci.

Możemy ograniczyć się do identyfikatora krotki. Sprawdza się przy układzie kolumnowym oraz – ze względu na oszczędność pamięci – gdy zwracanych jest bardzo dużo krotek.

## Ten sam problem po raz trzeci

Co zrobić, jeśli tablica z haszami nie mieści się w całości w pamięci operacyjnej?

Ponownie korzystamy z dwuetapowego algorytmu:

- w pierwszej fazie haszujemy za pomocą tej samej funkcji krotki z obu tabel,
- w drugiej fazie porównujemy krotki z obu tabel, które trafiły do tego samego kubelka.

## Ten sam problem po raz trzeci

Co zrobić, jeśli tablica z haszami nie mieści się w całości w pamięci operacyjnej?

Ponownie korzystamy z dwuetapowego algorytmu:

- w pierwszej fazie haszujemy za pomocą tej samej funkcji krotki z obu tabel,
- w drugiej fazie porównujemy krotki z obu tabel, które trafiły do tego samego kubłka.

```
for each i in {0, ..., #buckets - 1}
  for each r in bucketR[i]
    for each s in bucketS[i]
      if match (r, s)
        return (r, s)
```

## Ten sam problem po raz trzeci

Co zrobić, jeśli tablica z haszami nie mieści się w całości w pamięci operacyjnej?

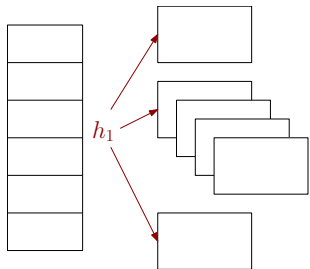
Ponownie korzystamy z dwuetapowego algorytmu:

- w pierwszej fazie haszujemy za pomocą tej samej funkcji krotki z obu tabel,
- w drugiej fazie porównujemy krotki z obu tabel, które trafiły do tego samego kubłka.

```
for each i in {0, ..., #buckets - 1}
  for each r in bucketR[i]
    for each s in bucketS[i]
      if match (r, s)
        return (r, s)
```

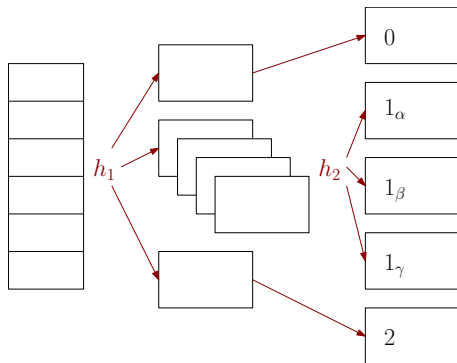
Co się dzieje, jeśli kubłki nie mieszczą się w pamięci operacyjnej?

# Wystarczy rekurencja

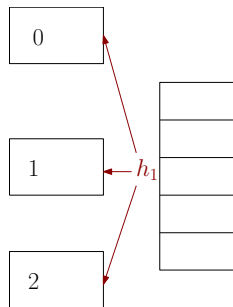
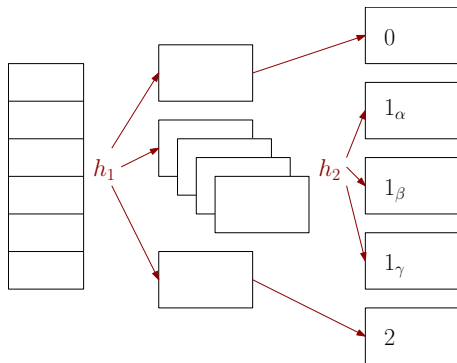




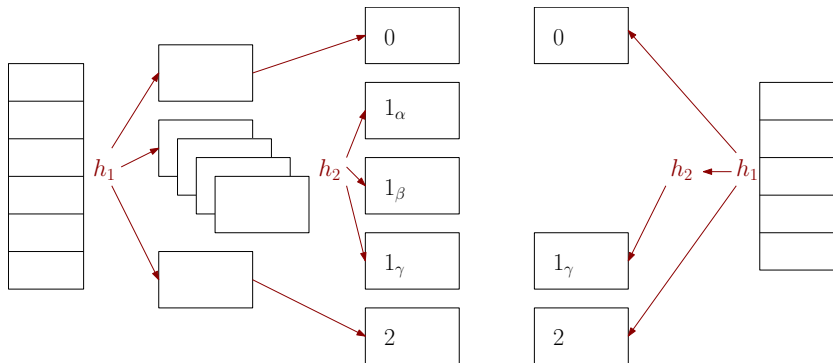
# Wystarczy rekurencja



# Wystarczy rekurencja



# Wystarczy rekurencja



# Analiza kosztu

Jeśli dysponujemy odpowiednią liczbą buforów, to koszt łączenia przez haszowanie wynosi  $3(M + N)$ :

- faza podziału wymaga dla każdej krotki z każdej tabeli pojedynczego odczytu i zapisu, czyli  $2(M + N)$  I/O,
- faza próbkowania wymaga odczytu krotek z obu tabel, czyli  $M + N$  I/O.

Dla przykładowych danych otrzymujemy  $3(M + N) = 4,500$  I/O, co przekłada się na czas wykonania ok. 0,45 sekundy.